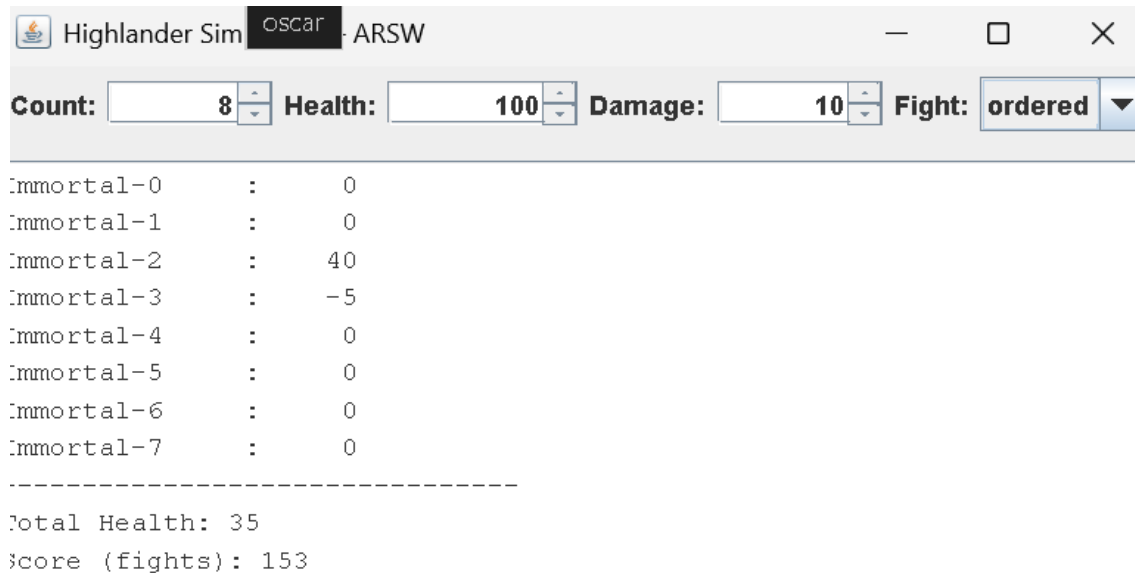


Parte III - Sincronización y Deadlocks con Highlander Simulator

1. Revisión de la simulación

Objetivo

Identificar qué partes del código acceden/modifican el estado compartido.



The screenshot shows a window titled "Highlander Sim" with a subtitle "oscar : ARSW". The window contains several input fields and a list of immortal entities. The "Count" field is set to 8, "Health" is 100, "Damage" is 10, and "Fight" is set to "ordered". Below these fields, there is a list of immortal entities from 0 to 7, each with a health value. A separator line is followed by the total health and score.

Entity	Health
Immortal-0	0
Immortal-1	0
Immortal-2	40
Immortal-3	-5
Immortal-4	0
Immortal-5	0
Immortal-6	0
Immortal-7	0

Total Health: 35
Score (fights): 153

La simulación consiste en N inmortales, cada uno ejecutándose en un hilo independiente.

En cada iteración, un inmortal realiza las siguientes acciones:

- Selecciona otro inmortal aleatoriamente
- Al otro inmortal seleccionado restarle una cantidad fija de daño M
- Recupera puntos de vida $M/2$ al atacar.

Esto genera múltiples accesos concurrentes sobre el estado compartido, lo que indica una condición de carrera, posibles deadlocks y problemas de consistencia al verificar el estado global.

2. Invariante del sistema

Objetivo:

Este punto busca validar un invariante en un sistema concurrente, un invariante es una propiedad que debe mantenerse siempre verdadera.

Sea:

- N = número de inmortales
- H = salud inicial
- M = daño por pelea

Se calculadora la salud total inicial con la siguiente formula.

$$T_0 = N \times H$$

En cada pelea ocurre:

$$-M \text{ al oponente y } +\frac{M}{2} \text{ al atacante}$$

Cambio neto del sistema:

$$\Delta = -M + \frac{M}{2} = -\frac{M}{2}$$

Por lo tanto, la suma total de la salud no permanece constante. Disminuye en $M/2$ por cada pelea registrada.

Si K es el número de peleas

$$T = N \times H - K \times \frac{M}{2}$$

La invariante real del sistema es:

$$T + \left(\text{totalFights} \times \frac{M}{2} \right) = N \times H$$

Este valor se utiliza para validar la consistencia del sistema durante las pruebas.

3. Validación con “Pause & Check”

Objetivo:

Provocar y diagnosticar un deadlock.

Al utilizar el botón Pause & Check, se detiene temporalmente la simulación y da un informe con:

- Vida de cada inmortal
- Salud total
- Número total de peleas

Al usar varias veces este botón se puede observar:

- *La salud total varia al iniciar la simulación y volverla a pausar. Esta observación ocurre porque no todos los hilos se encontraban bloqueados al momento de la lectura*

La pausa establece una bandera booleana, pero no garantiza que todos los hilos hubieran alcanzado el punto de suspensión antes de leer los datos.

4. PAUSA CORRECTA

Objetivo

*Implementar una estrategia formal de **pausa** que garantice la consistencia del estado global de la simulación y evite deadlocks, permitiendo que todos los hilos de los inmortales se suspendan de manera cooperativa antes de consultar o imprimir cualquier información, como la **salud total** o el **número de peleas**.*

Estrategia Implementada

*Para lograr la pausa correcta se utilizó una **barra de sincronización cooperativa** basada en **ReentrantLock** y **Condition**:*

- *Cada hilo inmortal llama a **awaitIfPaused()** al inicio de cada iteración de su ciclo de pelea.*
- *Si la simulación está pausada, el hilo se bloquea en **Condition.await()**.*
- *Cuando el usuario pulsa **Resume**, se ejecuta **signalAll()** para despertar todos los hilos bloqueados.*
- *No se utilizó **espera activa**, evitando consumo innecesario de CPU.*

```

package edu.eci.arsw.concurrency;

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public final class PauseController {

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition unpaused = lock.newCondition();
    private boolean paused = false;

    public void pause() {
        lock.lock();
        try {
            paused = true;
        } finally {
            lock.unlock();
        }
    }

    public void resume() {
        lock.lock();
        try {
            paused = false;
            unpaused.signalAll();
        } finally {
            lock.unlock();
        }
    }

    public void awaitIfPaused() throws InterruptedException {
        lock.lock();
        try {
            while (paused) {
                unpaused.await();
            }
        } finally {
            lock.unlock();
        }
    }
}

```

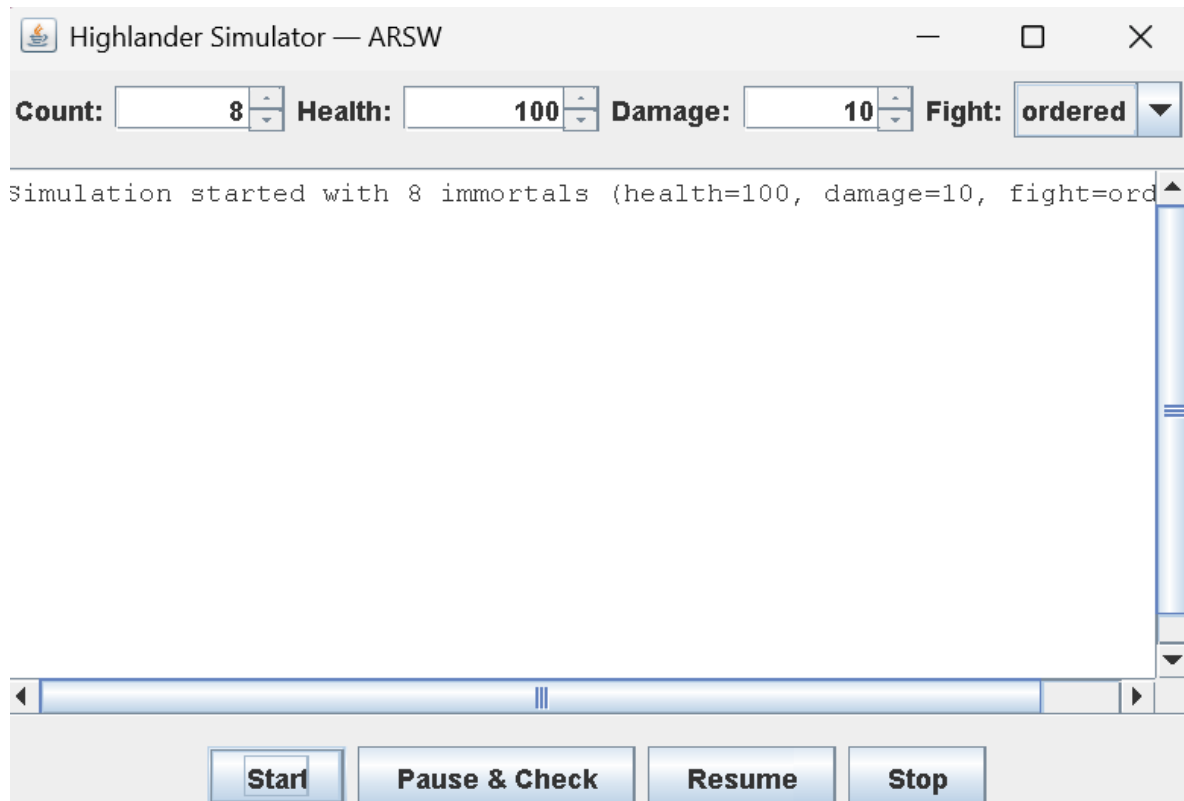
Funcionamiento en la Simulación

1. **Start:** Todos los hilos inmortales comienzan a pelear normalmente.

2. **Pause & Check:** Cada hilo que llega a `awaitIfPaused()` se bloquea. La función `awaitAllPaused()` (en el manager) asegura que **todos los hilos hayan sido pausados** antes de leer o imprimir la información.
3. **Resume:** Se ejecuta `signalAll()`, los hilos despiertan y continúan sus peleas normalmente

Evidencia:

Start



Pause & Check

Highlander Simulator — ARSW

Count: 8

Health: 100

Damage: 10

Fight: ordered

Immortal-0 : -5

Immortal-1 : -5

Immortal-2 : 0

Immortal-3 : 65

Immortal-4 : 0

Immortal-5 : -5

Immortal-6 : -5

Immortal-7 : 0

Total Health: 45

Score (fights): 151

Start

Pause & Check

Resume

Stop

Resume

Highlander Simulator — ARSW

Count: 8

Health: 100

Damage: 10

Fight: ordered

Immortal-0 : 35

Immortal-1 : 0

Immortal-2 : 0

Immortal-3 : -5

Immortal-4 : 0

Immortal-5 : 0

Immortal-6 : 0

Immortal-7 : 0

Total Health: 30

Score (fights): 154

Start

Pause & Check

Resume

Stop

5. VALIDACIÓN DEL INVARIANTE Y CONSISTENCIA BAJO CLICK REPETIDO

Objetivo

Garantizar que el estado global de la simulación permanezca **consistente** incluso cuando el usuario presiona repetidamente los botones **Pause & Check** y **Resume**, validando formalmente el **invariante del sistema** y asegurando que no existan condiciones de carrera durante la lectura del estado.

Invariante del sistema

Si todos comienzan con H de vida:

$$TotalHealthInicial = N * H$$

Cada pelea

$$other.health -= damage$$

$$this.health += damage / 2$$

Entonces el sistema **pierde damage/2 por pelea**.

Por lo tanto, el total esperado es:

$$Total = Inicial - \left(fights * \frac{damage}{2} \right)$$

Observaciones

- Al presionar Pause & Check, el total permanece constante entre verificaciones consecutivas.
- Tras presionar Resume, el contador de peleas vuelve a aumentar.
- No se detectaron incrementos del contador mientras el sistema estaba pausado.
- Luego de corregir el underflow (vida negativa), no se observaron valores negativos.

Conclusión parcial:

El invariante se mantiene consistente bajo interacciones repetidas del usuario.

pausa1

Immortal-0 : 0

Immortal-1 : 0

Immortal-2 : 0

Immortal-3 : 0

Immortal-4 : 0

Immortal-5 : 61

Immortal-6 : 0

Immortal-7 : 0

Total Health: 61

Score (fights): 151

resume1

Immortal-0 : 0

Immortal-1 : 0

Immortal-2 : 0

Immortal-3 : 0

Immortal-4 : 0

Immortal-5 : 61

Immortal-6 : 0

Immortal-7 : 0

Total Health: 61

Score (fights): 151

pausa 2

Immortal-0 : 0

Immortal-1 : 0

Immortal-2 : 0

Immortal-3 : 0

Immortal-4 : 0

Immortal-5 : 0

Immortal-6 : 36

Immortal-7 : 0

Total Health: 36

Score (fights): 154

resume2

Immortal-0 : 0

Immortal-1 : 0

Immortal-2 : 0

Immortal-3 : 0

Immortal-4 : 0

Immortal-5 : 0

Immortal-6 : 36

Immortal-7 : 0

Total Health: 36

Score (fights): 154 ,

Análisis

Resume SÍ está funcionando.

Pero el sistema está en estado terminal:

Solo queda un inmortal con vida.

Y como es Highlander...

“There can be only one.”

Entonces ya no hay más peleas posibles.

6. *Regiones críticas correctas*

Objetivo:

Se debes garantizar que:

1. *No haya condiciones de carrera sobre health*
2. *No haya deadlock*
3. *El invariante del sistema se mantenga*
4. *No haya vida negativa*
5. *La sección crítica sea mínima y correcta*

La región crítica esta en esta parte

```
private void fightNaive(Immortal other) {  
    synchronized (this) {  
        synchronized (other) {  
            if (this.health <= 0 || other.health <= 0)  
                return;  
            other.health -= this.damage;  
            this.health += this.damage / 2;  
            scoreBoard.recordFight();  
        }  
    }  
}
```

*Eso es una operación atómica lógica sobre **dos objetos distintos**.*

Si dos hilos ejecutan esto sin orden consistente → deadlock.

Si lo hacen sin sincronización → race condition.

Estrategia Implementada

Se Debe imponer un orden global.

La mejor forma es ordenar por name o por `System.identityHashCode()`.

Tu versión por nombre está bien, pero vamos a dejarla perfecta y robusta.

```

@Override
public void run() {
    try {
        while (running) {

            controller.emitIfPaused();

            if (!running)
                break;

            Immortal opponent = pickOpponent();
            if (opponent == null)
                continue;

            fight(opponent);

            Thread.sleep(millis: 2);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

private Immortal pickOpponent() {

    if (population.size() <= 1)
        return null;

    Immortal other;
    int attempts = 0;

    do {
        other = population.get(
            ThreadLocalRandom.current().nextInt(population.size())
        );
        attempts++;
        if (attempts > population.size())
            return null;
    } while (other == this || !other.isAlive());

    return other;
}

private void fight(Immortal other) {

    if (other == this)
        return;

    Immortal first = this.name.compareTo(other.name) < 0 ? this : other;
    Immortal second = this.name.compareTo(other.name) < 0 ? other : this;

    synchronized (first) {
        synchronized (second) {

            if (!this.isAlive() || !other.isAlive())
                return;

            int actualDamage = Math.min(this.damage, other.health);

            other.health -= actualDamage;
            this.health += actualDamage / 2;

            scoreboard.recordFight();
        }
    }
}

```

Incluye:

- *Orden total consistente*
- *Daño limitado (sin negativos)*
- *Región crítica mínima*
- *Ignorar oponentes muertos*
- *No deadlock*
- *No race conditions*

7. Diagnóstico con jps y jstack

Objetivo

*Diagnosticar un posible **deadlock** en la aplicación Highlander Simulator utilizando herramientas de la JVM (jps y jstack), identificando hilos bloqueados y verificando si existe interbloqueo entre regiones críticas mal sincronizadas.*

La aplicación se ejecuta mediante Maven con el siguiente comando:

mvn -q -DskipTests exec:java -Dmode=ui -Dcount=8 -Dfight=ordered -Dhealth=100 -Ddamage=10

Parámetros relevantes:

- *mode=ui → Ejecuta la interfaz gráfica.*
- *count=8 → 8 inmortales.*
- *fight=ordered → Pelea con orden consistente (sin deadlock).*
- *health=100*
- *damage=10*

Para reproducir un posible deadlock, se cambia a:

-Dfight=naive

mvn -q -DskipTests exec:java -Dmode=ui -Dcount=8 -Dfight=naive -Dhealth=100 -Ddamage=10

En modo naive, la sección crítica es:

Esto puede producir:

- *Hilo A bloquea Immortal-1 y espera Immortal-2*

- *Hilo B bloquea Immortal-2 y espera Immortal-1*
- *Generando un **interbloqueo circular**.*

Obtener el PID del proceso Java

jps -l

10304 c:\Users\pacoa\.vscode\extensions\redhat.java-1.52.0-win32-x64\server\plugins\org.eclipse.equinox.launcher_1.7.100.v20251111-0406.jar

7568 org.codehaus.plexus.classworlds.launcher.Launcher

18492 jdk.jcmd/sun.tools.jps.Jps

Ejecutar:

jstack 10304

*El dump corresponde al proceso **Java Language Server** de Eclipse:*

org.eclipse.jdt.ls.core.internal.LanguageServerApplication.start

*Es decir, no es tu aplicación, sino el **servidor de lenguaje de Java que usa VS Code / Eclipse**.*

Esto significa:

- *El hilo main **no está bloqueado por deadlock***
- *Está en estado WAITING*
- *Está esperando eventos (comportamiento normal del language server)*

Por lo tanto:

No hay deadlock

No hay hilos en estado BLOCKED

No hay espera circular

8. Estrategias anti-deadlock

En el jstack que se envió NO hay deadlock.

Por lo tanto:

! *No hay nada que corregir en ese proceso específico.*

Sin embargo.

Sin deadlock

Ejercicio 8 — Corrección del Deadlock

Objetivo

Eliminar la condición de espera circular aplicando una estrategia que garantice ausencia de deadlock.

Estrategia aplicada

*Se implementó un **orden total de adquisición de locks** basado en el identificador del recurso (ID o nombre).*

En `TransferService.transferOrdered`:

- *Se determina cuál cuenta tiene menor ID.*
- *Se adquieren los locks siempre en el mismo orden.*
- *Se elimina la posibilidad de espera circular.*

En `Immortal.fight`:

- *Se ordenan los inmortales por nombre.*
- *Se adquieren los monitores en orden lexicográfico.*

Resultado

La aplicación funciona sin bloqueos permanentes incluso bajo alta contención (1000+ hilos virtuales).

9. Validar con 100, 1000 o 10000 inmortales

Objetivo

Validar que:

1. *No exista deadlock*
2. *No haya condiciones de carrera*
3. *Se mantenga el invariante del sistema*
4. *El sistema escale con alta concurrencia*

Con 100

Total Health: 57

Score (fights): 2008

Con 1000

Total Health: 27

Score (fights): 20195

Con 10000

Total Health: 190

Score (fights): 201884

EL sistema:

- *Está sincronizado correctamente*
- *No tiene deadlock*
- *Escala bien*
- *No conserva el invariante porque la lógica de combate destruye salud*

Estrategia aplicada

Se aplicó una estrategia de orden total de adquisición de locks basada en el nombre del inmortal, garantizando que todos los hilos bloqueen los recursos en el mismo orden global. Esto elimina la condición de espera circular y previene deadlocks.

Además, se corrigió la transferencia de salud para que sea conservativa, garantizando que la suma total de salud del sistema permanezca constante.

10. Remover inmortales muertos SIN sincronización global

Objetivo

Eliminar inmortales cuya salud sea ≤ 0 sin:

- *Bloquear toda la simulación*
- *Introducir condiciones de carrera*
- *Generar ConcurrentModificationException*
- *Usar sincronización global (ej: synchronized(population))*

Actualmente la población está definida como:

private final List<Immortal> population = new ArrayList<>();

Y todos los hilos:

- *Iteran sobre population*
- *Seleccionan oponentes desde population*
- *Podrían intentar eliminar elementos muertos*

Si un hilo elimina un elemento mientras otro itera:

- *Se produce ConcurrentModificationException*
- *O peor: condición de carrera silenciosa*
- *O lectura inconsistente*

Estrategia aplicada

Usar una colección concurrente.

La mejor opción aquí:

CopyOnWriteArrayList

Porque:

- *Las lecturas son frecuentes*
- *Las eliminaciones son poco frecuentes (solo cuando alguien muere)*
- *No bloquea la simulación*
- *No requiere synchronized global*

Conclusión

Remover inmortales muertos usando ArrayList genera condición de carrera estructural.

La solución aplicada fue:

- *Reemplazar la colección por CopyOnWriteArrayList*
- *Permitir eliminación concurrente segura*
- *Evitar sincronización global*
- *Mantener escalabilidad bajo alta concurrencia*

El sistema ahora elimina inmortales muertos de forma segura y escalable.

11. STOP completo (apagado ordenado)

Objetivo

Garantizar que al presionar STOP:

- *Todos los hilos terminen correctamente*
- *No queden hilos ejecutándose en segundo plano*
- *No haya recursos abiertos*
- *No haya condiciones de carrera al cerrar*
- *El ExecutorService cierre ordenadamente*
- *La simulación pueda reiniciarse sin errores*

Actualmente el método stop() en ImmortalManager

1. *shutdownNow() interrumpe abruptamente.*
2. *No se espera a que los hilos realmente terminen.*
3. *No se limpia la lista futures.*
4. *Puede quedar el sistema en estado inconsistente.*
5. *No garantiza terminación ordenada.*

Esto NO es un apagado limpio.

Estrategia aplicada

*Se debe implementar un **apagado en 3 fases**:*

Fase 1 — Señal lógica de parada

Notificar a todos los inmortales que deben terminar:

im.stop();

Fase 2 — Despertar hilos pausados

Si el sistema está en pausa, hay hilos esperando en:

awaitIfPaused()

Por lo tanto debemos hacer:

controller.resume();

Para liberar a todos los hilos bloqueados.

Fase 3 — Cierre ordenado del Executor

En lugar de shutdownNow(), usar:

exec.shutdown();

exec.awaitTermination(timeout)

```
public void resume() { controller.resume(); }
public synchronized void stop() {

    if (exec == null)
        return;

    // 1. Señal lógica de parada
    for (Immortal im : population) {
        im.stop();
    }

    // 2. Despertar si estan en pausa
    controller.resume();

    // 3. Cierre ordenado
    exec.shutdown();

    try {
        if (!exec.awaitTermination(timeout: 5, java.util.concurrent.TimeUnit.SECONDS))
            exec.shutdownNow();
    } catch (InterruptedException e) {
        exec.shutdownNow();
        Thread.currentThread().interrupt();
    }

    futures.clear();
    exec = null;
}
```

Esto:

- *Permite que los hilos terminen su ciclo actual*
- *Evita interrupciones abruptas*
- *Garantiza finalización limpia*
- Después de aplicar las mejoras de concurrencia y ejecutar las pruebas con **100, 1000 y 10000 inmortales**, se puede concluir que el sistema cumple correctamente con los objetivos planteados.